Automated Reasoning

UniVR - Dipartimento di Informatica

Tommaso Toffali

1° Semestre 2026/2027

Indice

I **5**

1 Introduction **5**
1.1 Why study Automated Reasoning? 5

2 Historical Foundations and Theoretical Limits **5**
2.1 The Hilbert-Turing Perspective 5
2.2 Practical Applications of Automated Reasoning 6

3 Core Problems in Automated Reasoning **6**
3.1 Satisfiability problems 6
3.2 Validity problems 7
3.3 Logical Consequence Problems 7

4 Formal Language and Semantics **7**
4.1 Logical Language: First-Order Logic 7
4.2 Signature and Universe of Discourse 7

5 Syntactic Foundations: Terms and Formulas **9**
5.0.1 Terms: The building blocks 9
5.0.2 From terms to Atomic Formulas 9
5.1 Literals: Positive and Negative Atomic Formulas 10
5.2 Logical Connectives 10
5.3 Formulas: The Complete Syntax 10
5.4 Quantifiers: Universality and Existence 11
5.5 Illustrative Example: Geometric Axioms 11
5.6 Closed Formulas and Variable Scope 11

II Semantic Foundations **11**

6 Interpretations and Model **12**
6.1 The concept of Interpretation 12
6.2 Example: Interpreting a Formula over Natural Numbers 12
6.3 Semantic Evaluation 13
6.4 Formal Verification of Satisfaction 13
6.5 Formal Definition of Interpretation 13

7 Satisfaction Relation **14**
7.1 The satisfaction relation 14
7.2 Interpretation of Terms 14
7.3 Satisfaction for Atomic Formulas 14
7.4 Satisfaction for Logical Connectives 15
7.5 Example: Quantifier Evaluation 15

8 Semantic Properties of Formulas **16**
8.1 Satisfiability . . . 16
8.2 Validity . . . 16
8.3 Unsatisfiability . . . 16
8.4 Invalidity . . . 16
8.5 Relationship Between Semantic Properties . . . 16

9 Logical Consequence **17**
9.1 The Three Fundamental Query Types . . . 17
9.2 Definition of Logical Consequence . . . 17
9.3 The deduction Theorem . . . 18
9.4 Notational Ambiguity of $\models$ . . . 18
9.5 Reduction to Unsatisfiability . . . 18

10 Refutational Reasoning **19**
10.1 Proof by Contradiction . . . 19
10.2 The refutational Framework . . . 19

III Free Variables and Satisfiability **20**

11 Satisfiability of Formulas with Free Variables **20**
11.1 The problem of Free Variables . . . 20
11.2 Motivating Example . . . 20
11.3 Semantic Evaluation with Free Variables . . . 21
11.4 Existential Closure . . . 21
11.5 Satisfiability to Open Formulas . . . 22
11.6 Universal Closure and Validity . . . 22

12 Decidability and Semidecidability in First-Order Logic **23**
12.1 Computational Complexity of First-Order Logic . . . 23
12.2 Review: Semi-Decision Procedures . . . 23
12.3 The need for Restricted Theories . . . 24

13 First-Order Theories **24**
13.1 Definition of a First-Order Theory . . . 24
13.2 The theory of Equality . . . 24
13.3 Axioms of Equality . . . 24
13.4 Models of a Theory . . . 25
13.5 Satisfiability and Validity Relative to a Theory . . . 25
13.6 Example: Satisfiability vs. $\mathcal{T}_E$-Satisfiability . . . 25

14 Quantifiers-Free Fragments and Decidability **26**
14.1 The Quantifier-Free Fragment of $\mathcal{T}_E$ . . . 26
14.2 Normal Forms . . . 26
14.2.1 Negation Normal Form (NNF) . . . 27
14.2.2 Disjunctive Normal Form (DNF) . . . 27
14.3 Conversion to DNF . . . 27
14.4 Decision Procedure for Quantifier-Free $\mathcal{T}_E$ . . . 28
14.5 Elimination of Non-Equality Predicates . . . 28
14.5.1 The Transformation . . . 28

14.5.2 Satisfiability Preservation 29
14.5.3 Example: The Need for Cardinality Restriction 29
14.6 Treatment of Free Variables 29

15 The Congruence Closure Algrithm **30**
15.1 Motivating Example 30
15.2 The congruence closure Decision Procedure 30
15.3 Equivalence Relations and Partitions 31

IV Decision Procedures for Quantifier-Free Theories 31

16 The Congruence Closure Algorithm **32**
16.1 Computing Congruence Closure 32
16.1.1 Input Problem Structure 32
16.1.2 Equivalence relations and Representatives 32
16.1.3 Disjointness of Equivalence Classes 32
16.1.4 Example: Equivalence Closure 33
16.1.5 Understanding the $\subseteq$-Smallest Property 33
16.1.6 Example: Refinement of Equivalence Relations 34
16.2 The Congruence Closure Decision Procedure 35
16.2.1 Algorithm Description 35
16.2.2 Correctness of the Algorithm 36
16.3 Examples: Manual Execution 36
16.3.1 Notation and Setup 37
16.3.2 Example 1 37
16.3.3 Example 2 37
16.3.4 Step-by-Step Execution for Example 1 38

17 Implementation of Congruence Closure on a DAG **39**
17.1 DAG Representation 39
17.2 Basic Operations 40
17.2.1 The NODE Function 40
17.2.2 The FIND Operation 40
17.2.3 The UNION Operation 41
17.2.4 The CCPAR Function 41
17.2.5 The CONGRUENT Function 41
17.2.6 The MERGE Function 42
17.3 The Complete Congruence Closure Algorithm 43
17.3.1 Algorithm 43
17.3.2 Complexity 43

18 Worked Examples of Congruence Closure **44**
18.1 Example 1: Simple Commutativity 44
18.2 Example 2: Cyclic Function Application with Disequality 45

19 Complexity Improvements and Optimiazions **48**
19.1 Hisotical Development 48
19.1.1 Nelson-Oppen Algorithm (1980) 48
19.1.2 Downey-Sethi-Tarjan Improvements (1980-1982) 49
19.2 Key Optimization 49
19.2.1 Optimiazion 1: Avoiding Recursion in FIND 49
19.2.2 Optimiazion 2: Union by Size Heuristic 49
19.3 Implementation Note 50
19.4 Practical Impact 50

Parte I

1 Introduction

1.1 Why study Automated Reasoning?

Automated Reasoning is a foundamental field of Artificial Intelligence concerned with the design and implementation of computionals procedures that enable machines to solve problems formulated in logical languages. This procedures involve **logical inference and search**, allowing system to reason in a deductive manner throught mechanical process.

Since the inception of Artificial intelligence at the Dartmouth Conference in 1956, one of the core challenges has been to demonstrate machine intelligence throught the automated proving of mathematical and logical theorems. This capability represents a cornerstone of symbolic AI, where knowledge is rapresented symbolically and stored in computer memory, and systematic procedures are applied to solve complex problems.

Teorema 1.1 (Theorem Proving)

Traditionally require human-level mathematical reasoning and insight. The goal of automated reasoning is to mecanize this process, creating algorithms and systems capable of performing logical deduction without human intervention.

A logical language consist of formal symbols and syntactic rules. Within the symbolic approach to artificial intelligence, knowledge is encoded using these symbols, which are stored in computer memory. The system then applies a sequence of mechanical steps to manipulate these symbols and arrive at solutions to given problems.

2 Historical Foundations and Theoretical Limits

2.1 The Hilbert-Turing Perspective

The foundations of automated reasoning can be traced to fundamental questions in mathematical logic and computability. Since Alan Turing's seminal work on Turing machines in 1936, researchers have pursued David Hilbert's 1900 challenge: is it possible to develop a mechanical procedure-executed deterministically by human computor-capable of solving any mathematical problem?

The Entscheidungsproblem: A central question that emerged from this pursuit asks whether it is possible to define a mechanical method to detrmine the validity of formulas in first-order logic.

Turing demostrated that the halting problem is undecidable using Turing machines. This fundamental result can be reduced to the validity problem in first-order logic, providing a negative answer to Hilbert's challenge. This undecidability result

forms one of the core theoretical foundations of automated reasoning, estabilishing fundamental limits on what can be mechanically computed.

2.2 Practical Applications of Automated Reasoning

Automated reasoning find application across multiple domains requiring formal verification and systematic problem-solving:

- **Planning**: Given an initial state, goal state, and formal model of agent actions, determine a sequence of moves that transforms the initial state into the goal state (utilizing SAT solvers)
- **Scheduling**: Generate temporal arrangements of tasks that stisfy complex temporal and resource constraints
- **Software Analysis and Verfication**: Formal methods for ensuring program correctenss, including:
  - Provable privacy guarantees in security protocols
  - Verification of distributed algorithms
  - Analysis of randomized algorithms and their probabilistic properties
- **Hybrid Approaches**: Integration of automated reasoning with machine learning techniques for enhanced problem-solving capabilities

These applications demonstrate the practical utility of automated reasoning in solving complex real-world problems that require rigouros logical analysis and systematic exploration of solution spaces.

3 Core Problems in Automated Reasoning

Automated reasoning fundamentally addresses three interconnected classes of decision problems in mathematical logic, each representing a distinct challenge in determining the truth properties of logical formulas.

The Three Core Problems in Automated Reasoning

1. **Satisfiability (SAT)**: Does there exist a model?
2. **Validity**: Is the formula true in all models?
3. **Logical Consequence**: Does the conclusion follow from premises?

3.1 Satisfiability problems

A satisfiability problem (SAT) concerns determining whether a given logical formula has a truth assignment that makes it true. Formally, for a formula ϕ with free variables, we seek to determine if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models \phi$. This represent the existence problem - does there exists a model making the formula true?

3.2 Validity problems

In contrast to satisfiability, validity problems ask whether a formula holds true under all possible interpretations. A formula ϕ is valid (denoted $\models \phi$) if every possible truth assignment to its variables result in ϕ evaluating to true. This correspond to the unversal quantification over all possible models - the formula must hold in every conceivable scenario.

3.3 Logical Consequence Problems

The logical consequence problem represents a generalization of validity, asking whether a conclusion necessarly follows from given permises. Given a set of assumptions $\Gamma = \{\phi_1, \phi_2, \ldots, \phi_n\}$ and a conjecture ψ, we determine if $\Gamma \models \psi$ (i.e., if ψ is true in every model where all formulas in Γ are true). This captures the essence of logical deduction - what conclusions are forced by the given information.

These three problems form the foundation of automated reasoning systems, with satisfiability serving as the most tractable and finding widespread application in pratical problem-solving contexts.

4 Formal Language and Semantics

4.1 Logical Language: First-Order Logic

The formal language employed in automated reasoning is first-order logic (FOL), a substantial extension of propositional logic that incorporates quantification over individual elements.

First-Order Logic Components

First-order logic achieves greater expressive power by introducting:

- *Individual variables* ranging over domain elements
- *Function symbols* for constructing complex terms
- *Predicate symbols* for expressing relations between elements
- *Quantifiers* $\forall, \exists$ enabling statements about all or some elements

This enhanced expressiveness allows first-order logic to formalize mathematical structures, relationship, and properties in a manner impossible in propositional logic, which can only express truth-functional combinations of atomic propositions

4.2 Signature and Universe of Discourse

A signature Σ provied the vocabulary for interpreting formulas within a specific domain. Formally, a signature is a triple $\Sigma = \langle \mathcal{S}, \mathcal{F}, \mathcal{P} \rangle$ where:

- $\mathcal{S}$ is a set of sorts (disjoint domains of interpretation)
- $\mathcal{F}$ is a set of function symbols, each with an arity specifying the number of arguments

- $\mathcal{P}$ is a set of predicate symbols, which we use to write formulas, each with an associated arity

The signature defines the syntactic elements available for constructing formulas, while the universe of discourse (or domain) provides the semantic interpretation. Sorts correspond to types in programming languages, enabling the formal specification of heterogeneous domains with different categories of objects.

The signature $\Sigma = \langle \mathcal{S}, \mathcal{F}, \mathcal{P} \rangle$ cna be further decomposed to provide a complete syntactic framework. The set of function symbols $\mathcal{F}$ is typically partitioned into two distinct subset:

- **Constant symbols** $\mathcal{C} \subseteq \mathcal{F}$: These are function symbols of arity 0, representing named individuals in the domain. Constants have fixed interpretations and serve as proper names for specific domain elements
- **Proper function symbols** $\mathcal{F} \setminus \mathcal{C}$: These represent actual functions mapping tuples of domain elements to domain elements, each with arity $n \geq 1$.

Formally, a function symbol $f \in \mathcal{F}$ has type $s_1 \times s_2 \times \ldots \times s_n \to s$, where $s_1, \ldots, s_n, s \in \mathcal{S}$ are sorts indicating the domain and codomain of the function. For example:

- $c :\to \texttt{int}$ (a costant symbol denoting a specific integer)
- $p :\to \texttt{color}$ (a costant symbol denoting a specific color)
- $f : \texttt{int} \to \texttt{int}$ (a binary function like addition)

This decomposition enables precise specification of both individual elements (via constants) and functional relationships (via proper function symbol) within the formal language.

This complete signature structure $\langle \mathcal{S}, \mathcal{C} \cup (\mathcal{F} \setminus \mathcal{C}) \cup \mathcal{P} \rangle$ provides the full syntactic foundation for expressing complex logical statements about structured domains.

Constant can be viewed as function symbols of arity - functions that require no arguments and directly denote specific domain elements. Additionally, to construct quantified formulas and express generality, we require a countably infinte set of variables $X = \{x, y, z, \ldots\}$ disjoint from the signature symbols.

Thus, the function symbols may be comprehensively categorized as $\mathcal{F} = \mathcal{C} \cup (\mathcal{F} \setminus \mathcal{C}) \cup \mathcal{P}$, where $\mathcal{P}$ represents the set of predicate symbols. Each predicate symbol $p \in \mathcal{P}$ has arity $n \geq 0$ and type $s_1 \times s_2 \times \ldots \times s_n$, where the final sort represents the truth value (typically a boolean sort).

Predicate symbols differ fundamentally from function symbols in that they represent relations rather than functions - they evaluate to truth values rather than domain elements. For example:

- *even*: $\texttt{int} \to \texttt{bool}$ (unary predicate testing parity)
- *less*: $\texttt{int} \times \texttt{int} \to \texttt{bool}$ (binary relation for ordering)

The complete singature structure $\langle \mathcal{S}, \mathcal{C} \cup (\mathcal{F} \setminus \mathcal{C}) \cup \mathcal{P} \rangle$ provides the full syntactic foundation for expressing complex logical statements about structured domains.

Constants can be viewed as function symbols of arity 0 - functions that require no arguments and directly denote specific domain elements. Additionally, to construct quantified formulas and express generality, we require a countably infinite set of variables $X = \{x, y, z, \ldots\}$ disjoint from the signature symbols.

5 Syntactic Foundations: Terms and Formulas

5.0.1 Terms: The building blocks

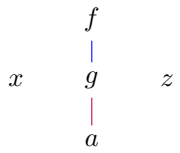
The most foundamental syntactic objects in first-order logic are **terms**, which represent individuals in the domain of discourse. The set of terms $\mathcal{T}_\Sigma(\mathcal{X})$ over signature Σ and variables $\mathcal{X}$ is inductively defined as the smallest set satisying:

- **Constants**: If $c \in \mathcal{C}$ is a constant symbol, then $c \in \mathcal{T}_\Sigma(\mathcal{X})$.
- **Variables**: If $x \in \mathcal{X}$ is a variable, then $x \in \mathcal{T}_\Sigma(\mathcal{X})$.
- **Function application**: If $f \in \mathcal{F}$ is a function symbol of arity $n \geq 1$ and $t_1, \ldots, t_n \in \mathcal{T}_\Sigma(\mathcal{X})$ are terms, then $f(t_1, \ldots, t_n) \in \mathcal{T}_\Sigma(\mathcal{X})$.

Terms can be conceptualized as finite trees where:

- Leaf nodes are constants or variables
- Internal nodes are function symbols applied to subterms

For example, the term $f(x, g(a), z)$ has the tree representation:



Terms may contain subterms, which correspond to subtrees in this representation. Complex terms are constructed recursively from simpler subterms, reflecting the hierarchical nature of the syntactic structure.

5.0.2 From terms to Atomic Formulas

Having established the notion of terms as representations of individuals, we now introduce atomic formulas to express relationships between these individuals. An **atomic formula** (or **atom**) is constructed by applying a predicate symbol to an appropriate number of terms.

Formally, if $P \in \mathcal{P}$ is a predicate symbol of arity $n \geq 0$ and $t_1, \ldots, t_n \in \mathcal{T}_\Sigma(\mathcal{X})$ are terms, then $P(t_1, \ldots, t_n)$ is an atomic formula. When $n = 0$, the atomic formula reduces to the predicate symbol itself P, which can be interpreted as a propositional constant.

The truth value of an atomic formula $P(t_1, \ldots, t_n)$ depends on the interpretation of the predicate P and the denotations of the terms $t_1, \ldots, t_n$ in a given model.

5.1 Literals: Positive and Negative Atomic Formulas

A **literal** is either an atomic formula (positive literal) or its negation (negative literal). Formally, if ϕ is an atomic formula, then:

- ϕ is a positive literal
- $\neg\phi$ is a negative literal

Literals represent the basic building blocks for constructing complex logical formulas and play a crucial role in automated reasoning system, particularly in resolution-based theorem proving.

5.2 Logical Connectives

Logical connectives provide the means to construct complex formulas simpler ones. These operators enable the expression of compound logical statements through systematic combination of atomic formulas and literals.

The primary logical connectives include:

- **Negation**($\neg$): Express logical denial or contradiction
- **Conjunction**($\wedge$): Represents logical conjunction (AND)
- **Disjunction**($\vee$): Represent logical disjunction (OR)
- **Implication**($\rightarrow$): Expresses logical implication (IF...THEN)
- **Biconditional**($\leftrightarrow$): Represent logical equivalence (IF AND ONLY IF)

5.3 Formulas: The Complete Syntax

The set of **formulas** (or **well-formed formulas**) $\mathcal{F}_\Sigma(\mathcal{X})$ over signature Σ and variables $\mathcal{X}$ is inductively defined as the smallest set satisying:

- **Atomic formulas**: If $P \in \mathcal{P}$ is a predicate symbol of arity $n \geq 0$ and $t_1, \ldots, t_n \in \mathcal{F}_\Sigma(\mathcal{X})$, then $P(t_1, \ldots, t_n) \in \mathcal{F}_\Sigma(\mathcal{X})$.
- **Negation**: If $\phi \in \mathcal{F}_\Sigma(\mathcal{X})$, then $(\neg\phi) \in \mathcal{F}_\Sigma(\mathcal{X})$.
- **Binary connectives**: If $\phi, \psi \in \mathcal{F}_\Sigma(\mathcal{X})$, then:
  - $(\phi \wedge \psi) \in \mathcal{F}_\Sigma(\mathcal{X})$ (conjunction)
  - $(\phi \vee \psi) \in \mathcal{F}_\Sigma(\mathcal{X})$ (disjunction)
  - $(\phi \rightarrow \psi) \in \mathcal{F}_\Sigma(\mathcal{X})$ (implication)
  - $(\phi \leftrightarrow \psi) \in \mathcal{F}_\Sigma(\mathcal{X})$ (biconditional)

This inductive definition ensures that all syntactically valid logical expressions can be systematically constructed from the basic building blocks of atomic formulas through the application of logical connectives.

5.4 Quantifiers: Universality and Existence

To complete the syntax of first-order logic, we introduce quantifiers that express generality and existence over the domain of discourse:

- **Universal quantifier** ($\forall$): If $\phi \in \mathcal{F}_\Sigma(\mathcal{X})$ is a formula and $x \in \mathcal{X}$ is a variable, then $(\forall x\phi) \in \mathcal{F}_\Sigma(\mathcal{X})$.
- **Existential quantifier** ($\exists$): If $\phi \in \mathcal{F}_\Sigma(\mathcal{X})$ is a formula and $x \in \mathcal{X}$ is a variable, then $(\exists x\phi) \in \mathcal{F}_\Sigma(\mathcal{X})$

Quantifiers bind variables within thei scope, enabling the expression of properties that hold for all elements of the domain (universal quantification) or for at least one element (existential quantification).

5.5 Illustrative Example: Geometric Axioms

Consider the following first-order formula expressing that any two distinct points determine a unique line:

$$\forall x \forall y((P(x) \wedge P(y) \wedge x \neq y) \rightarrow \exists z(L(z) \wedge Q(z,x,y) \wedge \forall w(L(w) \wedge Q(w,x,y) \rightarrow w = z)))$$

Where:

- P(x) denotes that x is a point
- L(z) denotes that z is a line
- Q(z,x,y) denotes that line z passes through points and x and y.
- $x \neq y$ expresses that points x and y are distinct

The axioms formalizes the geometric principle that given any two distinct points in space, there exists exactly one line passing through both points, capturing a foundamental property of Euclidean geometry within the framework of first-order logic.

5.6 Closed Formulas and Variable Scope

A **closed formula** (or **sentence**) is a formula containing no free variables - all variables are bound by quantifiers. A variable is **free** ina a formula if it appears outside the scope of any quantifier that binds it; conversely, a variable is **bound** if it appears within the scope of a quantifier.

Formulas may contain both free and bound occurences of variables. For proper semantica interpretation, variables must be **standardized apart**, meaning that no variable appears both free and bound in the same formula, and distinct quantifiers use distinct bound variables.

Parte II
Semantic Foundations

6 Interpretations and Model

6.1 The concept of Interpretation

To assign meaning to syntactic formulas in first-order logic, we require a formal notion of **interpretation**. An interpretation $\mathcal{I}$ provides a semantic mapping from the syntactic elements of a signature to concrete mathematical objects.

Definizione 6.1 (Interpretation Components)

In the unsorted case, an interpretation consists of:

- A non-empty set D called the **domain** (or universe of discourse)
- An interpretation function Φ that maps:
  - Each predicate symbol $P \in \mathcal{P}$ of arity n to a subset $\Phi(P) \subseteq D^n$ (representing an n-ary realtion over the domain)
  - Each function symbol $f \in \mathcal{F}$ of arity n to a function $\Phi(f) : D^n \to D$.
  - Each constants symbol $c \in \mathcal{C}$ to a specific element $\Phi(c) \in D$.

The interpretation function Φ bridges the gap between syntax and semantics, transforming abstract symbols into concrete mathematical entities.

6.2 Example: Interpreting a Formula over Natural Numbers

Consider the first-order formula:

$$\forall x(\neg P(x) \to P(f(x)))$$

The formula contains:

- A unary predicate symbol P
- A unary function symbol f
- A universally quantified variable x

Since the formula is closed (contains no free variables), its truth value depends entirely on the chosen interpretation.

Constructing an interpretation: Let us define an interpretation $\mathcal{I} = \langle D, \Phi \rangle$ where:

- **Domain**: $D = \mathbb{N}$ (the set of natural numbers)
- **Function interpretation**: $\Phi(f) : \mathbb{N} \to \mathbb{N}$ is the successor function, defined by $\Phi(f)(n) = n + 1$ for all $n \in \mathbb{N}$.
- **Predicate interpretation**: $\Phi(P) = \{0, 2, 4, 6, \ldots\} = \{2k \mid k \in \mathbb{N}\}$ (the set of even natural numbers)

6.3 Semantic Evaluation

Under this interpretation $\mathcal{I}$, the formula $\forall x(\neg P(x) \to P(f(x)))$ receives the following semantic reading:

For all natural numbers x, if x is not even, then the successor of x is even.

To determine whether this formula is true in interpretation $\mathcal{I}$ we must verify whether the implication holds for every element of the domain:

- If x is odd (i.e., $x \notin \Phi(P)$), then $x + 1$ must be even (i.e., $x + 1 \in \Phi(P)$)

The statement is indeed true in the natural numbers: the successor of any odd number is even. Therefore, the formula is **satisfied** by interpretation $\mathcal{I}$, and we write $\mathcal{I} \models \forall x(\neg P(x) \to P(f(x)))$.

6.4 Formal Verification of Satisfaction

The formally verify that $\mathcal{I} \models \forall x(\neg P(x) \to P(f(x)))$, we must check that for all $n \in \mathbb{N}$, the interpretation satisfies the implication with the substitution $\{x \leftarrow n\}$.

By the semantics of implications, $\mathcal{I}$ satisfies $(\neg P(x) \to P(f(x)))$ with assignment $\{x \leftarrow n\}$ if and only if:

- Either $\mathcal{I}$ does not satisfy $\neg P(x)$ with $\{x \leftarrow n\}$ (i.e., $\mathcal{I}$ satisfies $P(x)$ with $\{x \leftarrow n\}$), or
- $\mathcal{I}$ satisfies $P(f(x))$ with $\{x \leftarrow n\}$

Equivalently, for all $n \in \mathbb{N}$, either $n \in \Phi(P)$ or $\Phi(f)(n) \in \Phi(P)$.

Truth condition for atomic formulas: An atomic formula $P(t)$ is true under interpretation $\mathcal{I}$ if and only if the interpretation of the term t belongs to the interpretation of the predicate $\Phi(P)$..

In our example, for all $n \in \mathbb{N}$:

- Either $n \in \Phi(P)$ (meaning n is even), or
- $\Phi(f)(n) = n + 1 \in \Phi(P)$ (meaning the successor of n is even)

This condition holds for all natural numbers, confirming that the formula is satisfied by the interpretation.

6.5 Formal Definition of Interpretation

Having illustrated the concept through examples, we now provide the complete formal definition of an interpretation in first-order logic.

An **interpretation** $\mathcal{I}$ consists of a triple $\mathcal{I} = \langle D, \Phi, \beta \rangle$ where:

- **Domain**: D is a non-empty set representing the universe of discourse
- **Interpretation function** Φ maps signature symbols to semantic objects:
  - For each constant symbol $c \in \mathcal{C}$: $\Phi(c) = d \in D$.

- For each function symbol $f \in \mathcal{F}$ of arity $n \geq 1 : \Phi(f) : D^n \to D$, where D^n denotes the n-fold Cartesian product $D \times D \times \ldots \times D$.
- For each predicate symbol $P \in \mathcal{P}$ of arity $n \geq 0 : \Phi(P) \subseteq D^n$ (an n-ary relation over D)

- **Variable assignment** β maps each variable to a domain element:
  - $\beta : \mathcal{X} \to D$ such that $\beta(x) = d \in D$ for all variables $x \in \mathcal{X}$

Assignement update notation: Given an assignment β and a substitution $x \leftarrow d$, we define the updated assignment $\beta[x \leftarrow d]$ as:

$$\beta[x \leftarrow d](y) = \begin{cases} d & \text{if } y = x \\ \beta(y) & \text{otherwise} \end{cases}$$

This notation allows us to formally express the evaluation of formulas with different variable bindings, which is essential for defining the semantics of quantified formulas.

7 Satisfaction Relation

7.1 The satisfaction relation

Having defined interpretations and variable assignments, we now formalize the central semantica concept: when does an interpretation **satisfy** a formula? The satisfaction relation, denoted $\mathcal{I} \models_\beta \Phi$, specifies the conditions under which a formula Φ is true in interpretation $\mathcal{I}$ with variable assignment β.

Let F and G denote arbitraty formulas. The satisfaction realtion is defined inductively on the structure or formulas.

7.2 Interpretation of Terms

Before defining satisfaction for formulas, we must specify how terms are interpreted. Given an interpretation $\mathcal{I} = \langle D, \Phi, \beta \rangle$, the interpretation of a term t, denoted $\Phi_\beta(t)$, is defined recursively:

- **Constants**: $\Phi_\beta(c) = \Phi(c) \in D$
- **Variables**: $\Phi_\beta(x) = \beta(x) \in D$
- **Function application**: $\Phi_\beta(f(t_1, \ldots, t_n)) = \Phi(f)(\Phi(t_1), \ldots, \Phi_\beta(t_n)) \in D$

Each term evalueates to a specific element of the domain under a given interpretation.

7.3 Satisfaction for Atomic Formulas

Base case (atomic formulas): An interpretation $\mathcal{I}$ satisfies an atomic formula $P(t_1, \ldots, t_n)$ under assignment β if and only if the tuple of interpreted terms belongs to the interpretation of the predicate:

$$\mathcal{I} \models_\beta P(t_1, \ldots, t_n) \text{ iff } (\Phi_\beta(t_1), \ldots, \Phi_\beta(t_n)) \in \Phi(P)$$

7.4 Satisfaction for Logical Connectives

The satisfaction relation extends to compound fomrulas through the following inductive clauses. Note that all satisfaction relations are parametrized by the variable assignment β, written as $\models_\beta$

- **Negation**:
$$\mathcal{I} \models_\beta \neg F \text{ iff } \mathcal{I} \not\models_\beta F$$
- **Conjunction**:
$$\mathcal{I} \models_\beta (F \wedge G) \text{ iff } \mathcal{I} \models_\beta F \text{ and } \mathcal{I} \models_\beta G$$
- **Disjunction**:
$$\mathcal{I} \models_\beta (F \vee G) \text{ iff } \mathcal{I} \models_\beta F \text{ or } \mathcal{I} \models_\beta G$$
- **Implication**:
$$\mathcal{I} \models_\beta (F \to G) \text{ iff } \mathcal{I} \not\models_\beta F \text{ or } \mathcal{I} \models_\beta G$$
Equivalently: if $\mathcal{I} \models_\beta F$ then $\mathcal{I} \models_\beta G$.
- **Biconditional**:
$$\mathcal{I} \models_\beta (F \leftrightarrow G) \text{ iff } \mathcal{I} \models_\beta F \text{ if and only if } \mathcal{I} \models_\beta G$$
The formula $\exists x G$ is satisfied if G is satisfied under at least one assignment to x.

This completes the inductive definition of the satisfaction relation, providing a complete semantics for first-order logic.

7.5 Example: Quantifier Evaluation

Consider the formula $\forall x \exists y R(x, y)$, which can be read as "for all x, there exists y such that $x < y$".

Let $\mathcal{I} = \langle \mathbb{N}, \Phi \rangle$ where $\Phi(R) = \{(n, m) \in \mathbb{N} \times \mathbb{N} \mid n < m\}$ (the standard ordering on natural numbers).

To verify $\mathcal{I} \models \forall x \exists y R(x, y)$, we apply the semantic definitions:

$$\begin{aligned}
& \mathcal{I} \models \forall x\, \exists y\, R(x, y) \\
\iff & \text{ per ogni } n \in \mathbb{N},\ \mathcal{I} \models_{\beta[x \leftarrow n]} \exists y\, R(x, y) \\
\iff & \text{ per ogni } n \in \mathbb{N},\ \text{esiste } m \in \mathbb{N} \text{ tale che } \mathcal{I} \models_{\beta[x \leftarrow n,\, y \leftarrow m]} R(x, y) \\
\iff & \text{ per ogni } n \in \mathbb{N},\ \text{esiste } m \in \mathbb{N} \text{ tale che } (n, m) \in \Phi(R) \\
\iff & \text{ per ogni } n \in \mathbb{N},\ \text{esiste } m \in \mathbb{N} \text{ tale che } n < m
\end{aligned}$$

This statement is **false** in $\mathbb{N}$ because there is no natural number greater than all natural numbers. However, it would be true $\mathbb{Z}$ or $\mathbb{Q}$, demonstrating how truth depends on the chosen interpretaion.

8 Semantic Properties of Formulas

Having established the satisfaction relation, we now define fundamental semantic properties that classify formulas according to their to their truth behavior across different interpretations.

Definizione 8.1 (Semantic Properties of Formulas)

Satisfiability
A formula F is **satisfiable** if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models F$.

$$F \text{ is satisfiable } \Leftrightarrow \exists \mathcal{I} : \mathcal{I} \models F$$

Validity
A formula F is **valid** (or a **tautology**) if for all interpretations $\mathcal{I}$, we have $\mathcal{I} \models F$.

$$F \text{ is valid } \Leftrightarrow \forall \mathcal{I} : \mathcal{I} \models F$$

Unsatisfiability
A formula F is **unsatisfiable** if there exists no interpretation $\mathcal{I}$ such that $\mathcal{I} \models F$.

$$F \text{ is unsatisfiable } \Leftrightarrow \nexists \mathcal{I} : \mathcal{I} \models F$$

Invalidaity
A formula F is **invalid** if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \not\models F$.

$$F \text{ is invalid } \Leftrightarrow \exists \mathcal{I} : \mathcal{I} \not\models F$$

8.1 Satisfiability

A formula F is **satisfiable** if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models F$. In thi case, we say that $\mathcal{I}$ is a **model** of F.

8.2 Validity

We denote validity by $\models F$.

8.3 Unsatisfiability

Unsatisfiable formulas are false in every interpretation.

8.4 Invalidity

In this case, we say that $\mathcal{I}$ is a **countermodel** (or **counterexample**) for F.

8.5 Relationship Between Semantic Properties

These semantic properties are intimately related through negation:

Teorema 8.2 (Validity and Unsatisfiability)

A formula F is valid if and only if $\neg F$ is unsatisfiable.

Proof. Suppose F is valid. Then for all interpretations $\mathcal{I}$, we have $\mathcal{I} \models F$, which implies $\mathcal{I} \not\models \neg F$.
Therefore, $\neg F$ is unsatisfiable.

Conversely, if $\neg F$ is unsatisfiable, then for all interpretations $\mathcal{I}$, we have $\mathcal{I} \not\models \neg F$, which implies $\mathcal{I} \models F$. Therefore, F is valid.

Teorema 8.3 (Satisfiability and Invalidity)

A formula F is satisfiable if and only if $\neg F$ is invalid

Proof. F is satisfiable if and only if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models F$, which is equivalent to $\mathcal{I} \not\models \neg F$. This is precisely the condition for $\neg F$ to be invalid.

These relationships demonstrate the duality between validity and unsatisfiability, and between satisfiability and invalidity, providing a complete characterization of the semantic landscape of first-order logic.

9 Logical Consequence

9.1 The Three Fundamental Query Types

Automated reasoning system must be capable of resolving three fundamental types of semantic queries:

- **Satisfiability queries**: Is formula F satisfiable?
- **Validity queries**: Is formula F valid?
- **Logical consequence queries**: Does F follow logically from a set of hypotheses H?

9.2 Definition of Logical Consequence

Definizione 9.1 (Logical Consequence)

Let $H = \{\phi_1, \phi_2, \ldots, \phi_n\}$ be a set of formulas (the **hypotheses** or **premises**) and let F be a formula (the **conclusion**). We say that F is a **logical consequence** of H, written $H \models F$, if:

$$H \models F \Leftrightarrow \text{ for all interpretations } \mathcal{I}, \text{ if } \mathcal{I} \models H \text{ then } \mathcal{I} \models F$$

In other words, F is a logical consequence of H if every model of H is also a model of F. Here, $\mathcal{I} \models H$ means that $\mathcal{I}$ satisfies all formulas in H.

9.3 The deduction Theorem

The deduction theorem establishes a fundamental connection between logical consequence and validity, reducing the probem of checking entailment to the problem of checking validity.

Definizione 9.2 (Deduction theorem)

Let H be a set of formulas and F be a formula.
Then:

$$H \models F \text{ iff } \models (H \to F)$$

where $H \to F$ denotes the implication from the conjunction of all formulas in H to F

Proof. We prove both directions:
($\Rightarrow$) Assume $H \models F$. We must show that $H \to F$ is valid, i.e., for all interpretations $\mathcal{I}$, we have $\mathcal{I} \models H \to F$.

Let $\mathcal{I}$ be an arbitraty interpretation. By the semantics of implication, $\mathcal{I} \models H \to F$, holds if either:

- $\mathcal{I} \not\models H$ (the antecedent is false), or
- $\mathcal{I} \models F$ (the consequent is true)

If $\mathcal{I} \not\models H$, then $\mathcal{I} \models H \to F$ trivially. If $\mathcal{I} \models H$, then by the hypothesis $H \models F$, we have $\mathcal{I} \models F$, so $\mathcal{I} \models H \to F$. In both cases, $\mathcal{I} \models H \to F$.

($\Leftarrow$) Conversely, suppose $H \to F$ is valid. Then for all interpretations $\mathcal{I}$, we have $\mathcal{I} \models H \to F$. By the semantics of implications, for all $\mathcal{I}$, either $\mathcal{I} \not\models H$ or $\mathcal{I} \models F$. Therefore, for all $\mathcal{I}$ such that $\mathcal{I} \models H$, we must have $\mathcal{I} \models F$, which the definition of $H \models F$.

9.4 Notational Ambiguity of $\models$

The symbol $\models$ is used in three distinct but related contexts:

- **Satisfaction**: $\mathcal{I} \models F$ means interpretation $\mathcal{I}$ satisfies formula F.
- **Validity**: $\models F$ means formula F is valid (satisfied by all interpretations)
- **Entailment**: $H \models F$ means formula F is a logical consequence of the set of formulas H.

While this overloading may initially seem confusing, these three uses are smentically coherent and represent different aspects of the same fundamental semantic relation.

9.5 Reduction to Unsatisfiability

A fundamental result connects logical consequence to unsatisfiability, providing another characterization of entailment.

Teorema 9.3 (Jey Theorem: Reduction to Unsatisfiability)

Let H be a set of formulas and F be a formula. Then:

$$H \models F \text{ iff } H \cup \{\neg F\} \text{ is unsatisfiable}$$

Proof. We prove both directions:

($\Rightarrow$) Assume $H \models F$. We must show that $H \cup \{\neg F\}$ is unsatisfiable.

Suppose for contradiction that there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models H \cup \{\neg F\}$. Then $\mathcal{I} \models H$, and $\mathcal{I} \models \neg F$, which implies $\mathcal{I} \not\models F$. However, sincer $H \models F$ and $\mathcal{I} \models H$, we must have $\mathcal{I} \models F$, which is a contradiction. Therefore, $H \cup \{\neg F\}$ is unsatisfiable.

($\Leftarrow$) Assume $H \cup \{\neg F\}$ is unsatisfiable. We must show that $H \models F$.

Let $\mathcal{I}$ be any interpretation such that $\mathcal{I} \models H$. Since $H \cup \{\neg F\}$ is unsatisfiable, there is no interpretation that satisfies both H and $\neg F$. Therefore, $\mathcal{I} \not\models \neg F$, which implies $\mathcal{I} \models F$. Since this holds for all interpretations satisfying H, we conclude that $H \models F$.

10 Refutational Reasoning

10.1 Proof by Contradiction

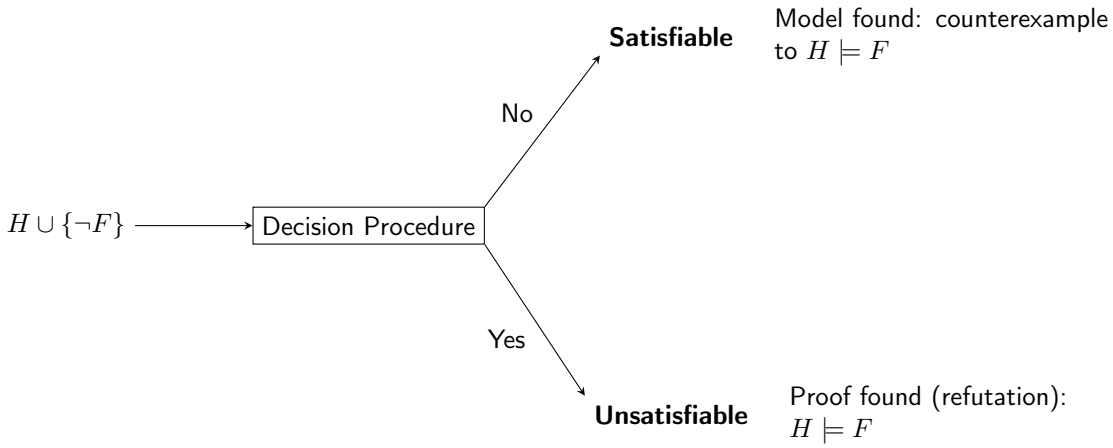
Definizione 10.1 (Refutational reasoning (Proof by contradiction))

Refutational reasoning (or **proof by refutation**) is a fundamental techinique in automated reasoning on the principle of *reductio ad absurdum*. To prove that a conclusion F follows from hupotheses H, we assume the negation of the conclusion and derive a contradiction.

1. **Goal**: Prove $H \models F$ (where H represents assumpions and F represents the conjecture)
2. **Method**: Show that $H \cup \{\neg F\}$ is unsatisfiable
3. **Conclusion**: If $H \cup \{\neg F\}$ is unsatisfiable, then $H \models F$ by the previous theorem

10.2 The refutational Framework

The automated reasoning process can be formalized as follows:



Two possible outcomes:

- **Unsatisfiable**: The procedure derives a contradiction, producing a **refutation** (proof) that $H \models F$.
- **Satisfiable**: The procedure finds a model $\mathcal{I}$ of $H \cup \{\neg F\}$, which serves as a **counterexample** showing that $H \not\models F$.

This refutational approach forms the foundation of modern automated theorem provers and SAT solvers, reducing the problem of proving logical consequence to the problem of detecting unsatisfiability.

Parte III
Free Variables and Satisfiability

11 Satisfiability of Formulas with Free Variables

11.1 The problem of Free Variables

In our previous definition of statisfiability, we considered the semantics of logical connectives and quantifiers, but we did not fully address the tratment of **free variables**. Recall that a variable can be either:

- **Bound**: The variable falls within the scope of a quantifier
- **Free**: The variable does not fall within the scope of any quantifier

The presence of free variables requires careful consideration when determining satisfiability, as their interpretation depends on the variable assignment β.

11.2 Motivating Example

Consider the following formula:

$$F : \forall x(f(x, x) = x \wedge R(x, y))$$

The formula contains:

- A bound variable x (qunatified by $\forall$)
- A free variable y (not quantified)

11.3 Semantic Evaluation with Free Variables

To determine whether an interpretation $\mathcal{I} = \langle D, \Phi \rangle$ with assignment β satisfies F, we write $\mathcal{I} \models_\beta F$ and proceed as follows:

By semantics of universal quantification, we need to check that for all $d \in D$:

$$\mathcal{I} \models_{\beta[x \leftarrow d]} (f(x, x) = x \wedge R(x, y))$$

This requires verifiying both conjuncts under the updated assignment $\beta[x \leftarrow d]$:

First conjuction: $\mathcal{I} \models_{\beta[x \leftarrow d]} f(x, x) = x$

This holds if and only if:

$$(\Phi(f)(d, d), d) \in \Phi(=)$$

where $\Phi(=)$ represents the identity relation. In other words, we require that $\Phi(f)(d, d) = d$ for all $d \in D$.

Second conjunct: $\mathcal{I} \models_{\beta[x \leftarrow d] R(x,y)}$

This holds if and only if:

$$(d, \beta(y)) \in \Phi(R)$$

Let $\beta(y) = d'$ for some $d' \in D$. Then we require $(d, d') \in \Phi(R)$ for all $d \in D$.

11.4 Existential Closure

The key observation is that the satisfiability of a formula with free variables is intimately connected to its **existential closure**.

Definizione 11.1 (Existential Closure)

The existential closure of a formula F, denoted $\exists^* F$, is obtained by adding an existential quantifier for every free variable in F.

For example, if F contains free variables $y_1, y_2, \ldots, y_n$ then:

$$\exists^* F = \exists y_1 \exists y_2 \ldots \exists y_n F$$

Returning to our example, if we consider the existential closure by quantifying the free variable y:

$$\exists y \forall x (f(x, x) = x \wedge R(x, y))$$

Under this interpretation, we need to verify that there exists some $d' \in D$ such that for all $d \in D$.

11.5 Satisfiability to Open Formulas

We now provide the complete definition of satisfiability that handles both closed and open formulas.

Definizione 11.2 (Satisfiability - Complete Definition)

A formula F is **satisfiable** if:

- **Case 1 (Closed formula)**: If F is a sentece (contains no free variables), then F is satisfiable if there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models F$.
- **Case 2 (Open formula)**: If F contains free variables, then F is satisfiable if there exists and interpretation $\mathcal{I}$ and a variable assignment β such that $\mathcal{I} \models_\beta F$.

Teorema 11.3 (Satifiability and Existential Closure)

A formula F with free variables is satisfiable if and only if its existential closure $\exists^* F$ is satisfiable

Proof. Let F be a formula with free variables $y_1, \ldots, y_n$ and let $\exists^* F = \exists y_1 \ldots \exists y_n F$.

($\Rightarrow$) Suppose F is satisfiable. Then there exists and interpretation $\mathcal{I}$ and assignment β such that $\mathcal{I} \models_\beta F$. Let $d_i = \beta(y_i)$ for $i = 1, \ldots, n$. Then by the semantics of existential quantification applied repeadetly, $\mathcal{I} \models \exists^* F$.

($\Leftarrow$) Suppose $\exists^* F$ is satisfiable. Then there exists an interpretation $\mathcal{I}$ such that $\mathcal{I} \models \exists y_1 \ldots \exists y_n F$. By the semantics of existential quantification, there exist $d_1, \ldots, d_2 \in D$ such that $\mathcal{I} \models_{\beta[y_1 \leftarrow d_1, \ldots, y_n \leftarrow d_n]}$.

This result demonstrates that when checking satisfiability of formulas with free variables, we can equivalentlòy check the satisfiability of their existential closure, treating free variables as existentailly qunatified.

11.6 Universal Closure and Validity

Dually, we can consider the **universal closure** of a formula with free variables.

Teorema 11.4 (Validity and Universal Closure)

A formula F with free variables is valid if and only if its universal closure $\forall^* F$ is valid.

The intuition is straightforwared: if F contains free variables, then F is valid if for all interpretations $\mathcal{I}$ and for all assignments β, we have $\mathcal{I} \models_\beta F$. This is precisely the condition for the univesal closure $\forall^* F$ to be valid.

12 Decidability and Semidecidability in First-Order Logic

12.1 Computational Complexity of First-Order Logic

While certain restricted fragments of first-order logic admit decision procedures, the general case present fundamental computational limitations:

Definizione 12.1 (Decidability in First-Order Logic)

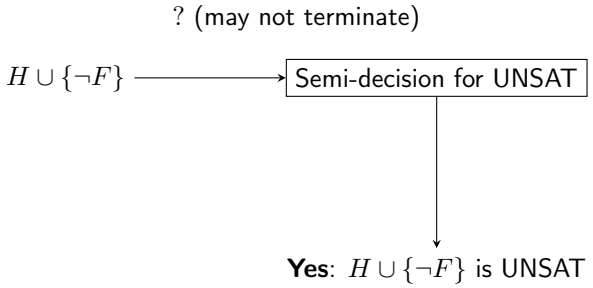
- **Unsatisfiability** in first-order logic is **semidecidable** (recursively enumerable)
- **Satisfiability** in first-order logic is **not even semidecidable**

Note. Recall that if a decision problem A is semidecidable and its complement $\neg A$ is also semidecidable, then A is decidable. The decision procedure can be obtained by running both semidecision procedures in parallel: if the instance belongs to A, the first procedure will eventually terminate with "yes"; if the instance belogs to $\neg A$, the second procedure will eventually terminate with "yes".

12.2 Review: Semi-Decision Procedures

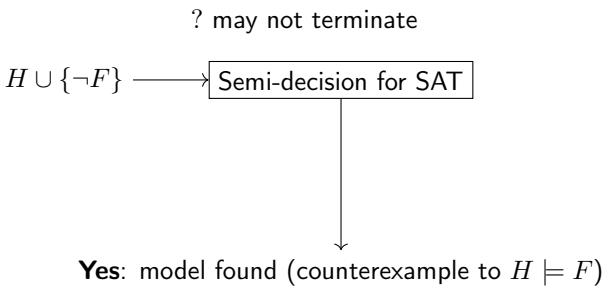
Consider the problem of determining whether $H \models F$ (logical consequence).

Semi-decision procedure for unsatisfiability:



If the procedure terminate with "yes", we have proven $H \models F$ by refutation. However, if $H \cup \{\neg F\}$ is satisfiable, the procedure may run indefinitely.

Semi-decision procedure for satisfiability



If this procedure terminates with "yes", we have found a countermodel showing $H \nvDash F$. However, such a procedure does not exists for general first-order logic.

12.3 The need for Restricted Theories

To obtain decision procedures, we must restrict out attention to **decidable fragments** of first-order logic. This motivates the study of **first-order theories** - logical systems with additional structure that enable algorithmic reasoning.

13 First-Order Theories

13.1 Definition of a First-Order Theory

Definizione 13.1 (First-Order Theory)

A **first-order theory** is a pair $\mathcal{T} = \langle \sigma, \mathcal{A} \rangle$ where:

- σ is a **signature** (defining the vocabulary of the theory)
- $\mathcal{A}$ is a set of **axioms** - closed formulas (sentences) that we assume to hold in all models of the theory

The axioms define properties of specific symbols in the signature, giving them fixed interpretations across all models of the theory.

13.2 The theory of Equality

A fundamental example is the **theory of equality** $\mathcal{T}_E = \langle \sigma_E, \mathcal{A}_E \rangle$.

Signature: The signature of the theory of equality is:

$$\sigma_E = \{=\} \cup \mathcal{C} \cup \mathcal{F} \cup \mathcal{P}$$

where:

- $=$ is the equality predicate (interpreted symbol)
- $\mathcal{C}$ is a set of constant symbols (uninterpreted)
- $\mathcal{F}$ is a set of function symbols (uninterpreted)
- $\mathcal{P}$ is a set of predicate symbols (uninterpreted)

Example signature: Consider $\sigma = \{a, b, f, Q, =\}$. The symbols a, b, f, Q are **free** (uninterpreted), whereas $=$ (equality) is **defined** by the axioms of the theory.

13.3 Axioms of Equality

The axiom set $\mathcal{A}_E$ consists of formulas that characterize equality as a congruence relation.

Equivalence relation axioms:

$$\begin{aligned} &\forall x\,(x = x) && \text{(Reflexivity)} \\ &\forall x\,\forall y\,(x = y \to y = x) && \text{(Symmetry)} \\ &\forall x\,\forall y\,\forall z\,((x = y \wedge y = z) \to x = z) && \text{(Transitivity)} \end{aligned}$$

These three axioms establish that equality is an equivalance relation.

Congruence axioms of function symbols: For each function symbol $f \in \mathcal{F}$ of arity n:

$$\forall \overrightarrow{x}\,\forall \overrightarrow{y}\,(\bigwedge_{i=1}^{n} x_i = y_i) \to f(x_1, \ldots, x_n) = f(y_1, \ldots, y_n)$$

where $\overrightarrow{x} = (x_1, \ldots, x_n)$ and $\overrightarrow{y} = (y_1, \ldots, y_n)$.

This axioms states that f respects equality: equal inputs produce equal outputs.

Congruence axioms for predicate sumbols: For each predicate symbol $R \in \mathcal{P}$ of arity n:

$$\forall \overrightarrow{x}\,\forall \overrightarrow{y}\,(\bigwedge_{i=1}^{n} x_i = y_i) \to (R(x_1, \ldots, x_n) \leftrightarrow R(y_1, \ldots, y_n))$$

This axiom states that R resprects equality: the truth value of R depends only on the equivalence classes of its arguments.

13.4 Models of a Theory

Definizione 13.2 ($\mathcal{T}$-Model)

A $\mathcal{T}$-**Model** is an interpretation $\mathcal{I}$ such that $\mathcal{I} \models \mathcal{A}$ (i.e., $\mathcal{I}$ satisfies all axiom in $\mathcal{A}$).

13.5 Satisfiability and Validity Relative to a Theory

Definizione 13.3 ($\mathcal{T}$-Satisfiability)

A formula G is $\mathcal{T}$-**satisfiable** if there exists a $\mathcal{T}$-model $\mathcal{I}$ such that $\mathcal{I} \models G$.

Definizione 13.4 ($\mathcal{T}$-Validity)

A formula G is $\mathcal{T}$-**valid** if for all $\mathcal{T}$-models $\mathcal{I}$, we have $\mathcal{I} \models G$.

Equivalently, G is a logical consequence of the axioms: $\mathcal{A} \models G$.

13.6 Example: Satisfiability vs. $\mathcal{T}_E$-Satisfiability

Consider the set of formulas:

$$S = \{P(a), \neg P(b), a = b\}$$

Claim: S is satisfiable in general first-order logic, but S is **not** $\mathcal{T}_E$**-satisfiable**.

Proof. **Satisfiability**: We can construct an interpretation $\mathcal{I}$ with domain $D = \{d_1, d_2\}$ where:

- $\Phi(a) = d_1, \Phi(b) = d_2$.
- $\Phi(P) = \{d_1\}$.
- $\Phi(=) = \{(d_1, d_1), (d_2, d_2), (d_1, d_2)(d_2, d_1)\}$ (universal relation)

Then $\mathcal{I} \models P(a), \mathcal{I} \models \neg P(b),$ and $\mathcal{I} \models a = b,$ so $\mathcal{I} \models S$.

$\mathcal{T}_E$**-Unsatisfiability**: In any $\mathcal{T}_E$-model, equality must satisfy the congruence axioms. If $\mathcal{I} \models a = b,$ and $\mathcal{I} \models P(a)$, then by the congruence axiom for P, we must have $\mathcal{I} \models P(b)$, contradicting $\mathcal{I} \models \neg P(b)$. Therefore, no $\mathcal{T}_E$-model satisfies S.

This example illustrate that satisfiability relative to a theory is more restrictive than general satisfiability, as models must respect the additional constraints imposed by the theory's axioms.

14 Quantifiers-Free Fragments and Decidability

14.1 The Quantifier-Free Fragment of $\mathcal{T}_E$

While the full theory of equality $\mathcal{T}_E$ is undecidable, its **quantifier-free fragment** admits a decsion procedure.

Definizione 14.1 (Quantifier-Free Fragment)

The **quantifier-free fragment** of first-order logic consists of all formulas that do not contain quantifiers ($\forall$ or $\exists$).

For a theory $\mathcal{T}$, the quantifier-free fragment allows quantifiers to appear only in the axioms, not in the input formulas to be checked for satisfiability.

Our goal is to develop a decision procedure for the quantifier-free fragment of $\mathcal{T}_E$. We will focus initially on deciding satisfiability for sets (or conjunctions) of literals, and then extend this to arbitrary quantifier-free formulas using normal forms.

14.2 Normal Forms

To handle arbitrary quantifier-free formulas, we introduce two important normal forms that facilitate systematic resoning.

14.2.1 Negation Normal Form (NNF)

Definizione 14.2 (Negation Normal Form)

A formula is in **Negation Normal Form (NNF)** if:

- The only logical connectives are $\wedge, \vee,$ and $, \neg$.
- Negation $(\neg)$ applies only to atomic formulas (atoms)

Any quantifier-free formula can be converted to NNF by applying the following equivalence preserving transformations:

$H \leftrightarrow G \equiv (H \rightarrow G) \wedge (G \rightarrow H)$	(Eliminate biconditional)
$H \rightarrow G \equiv \neg H \vee G$	(Eliminate implication)
$\neg(H \wedge G) \equiv \neg H \vee \neg G$	(De Morgan's law)
$\neg(H \vee G) \equiv \neg H \wedge \neg G$	(De Morgan's law)
$\neg\neg H \equiv H$	(Double negation elimination)

14.2.2 Disjunctive Normal Form (DNF)

Definizione 14.3 (Disjunctive Normal Form)

A formula is in **Disjunctive Normal Form (DNF)** if it is a disjunction of conjunctions of literals:

$$\bigvee_{i=1}^{k} \left(\bigwedge_{j=1}^{n_i} L_{i,j} \right)$$

where each $L_{i,j}$ is a literal (an atomic formula or its negation)

Equivalently, a DNF formula has the structure:

$$(L_{1,1} \wedge L_{1,2} \wedge \ldots L_{1,n_1}) \vee (L_{2,1} \wedge L_{2,2} \wedge \ldots \wedge L_{2,n_2}) \vee \ldots \vee (L_{k,1} \wedge L_{k,2} \wedge \ldots \wedge L_{k,n_k})$$

14.3 Conversion to DNF

Any quantifier-free formula F can be systematically converted to an equivalent DNF formula throught a two-stage process:

$$F \longrightarrow \boxed{\text{Convert to NNF}} \longrightarrow F' \longrightarrow \boxed{\text{Convert to DNF}} \longrightarrow F''$$

Stage 1: Convert F to NNF using the transformations above, obtaining F''.

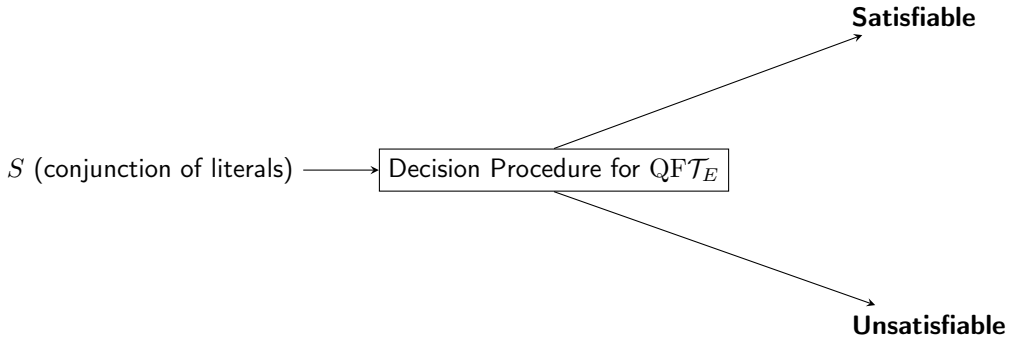
Stage 2: Convert F' to DNF by repeatedly applying the distributivity law:

$$H \wedge (G_1 \vee G_2) \equiv (H \wedge G_1) \vee (H \wedge G_2)$$

This transformation pushes conjunctions inward and disjunctions outward, resulting in a disjunction of conjunctions.

14.4 Decision Procedure for Quantifier-Free $\mathcal{T}_E$

The decision procedure operates on conjunctions of literals. Given a set (or conjunctions) S of literals:



Extension to arbitrary quantifier-free formulas: For a general quantifier-free formula F:

1. Convert F to NNF, obtaining F'.
2. Convert F' to DNF, otaining $F'' = \wedge_{i=1}^{k} \mathcal{C}_i$ where each $\mathcal{C}_i$ is a conjunction of literals
3. Apply the decision procedure to each conjunction $\mathcal{C}_i$.
4. F is satisfiable if and onlu if at least one $\mathcal{C}_i$ is satisfiable

This reduction allows us to decide satisfiability for arbitrary quantifier-free formulas by deciding satisfiability for each disjunct independently.

14.5 Elimination of Non-Equality Predicates

The decision procedure for quantifier-free $\mathcal{T}_E$ assumes that the only predicate symbol in the formula is equality (=). However, formulas may contain additional predicate symbols. We now show how to elinate such predicates throught a satisfiability-preserving transformation.

14.5.1 The Transformation

Definizione 14.4 (Predicate Elimination)

Given an occurence of a predicate symbol P of arity n applied to terms $t_1, \ldots, t_n$, we eliminate P by introducing:

- A fresh function symbol f_P (one for each predicate symbol $P \neq =$)
- A fresh constant symbol $\bullet$ (the same constant for all predicates)

The transformation replaces predicate applications with equality constraints:

$$P(t_1, \ldots, t_n) \mapsto f_P(t_1, \ldots, t_n) = \bullet$$

$$\neg P(t_1, \ldots, t_n) \mapsto f_P(t_1, \ldots, t_n) \neq \bullet$$

Intuition: The predicate $P(t_1, \ldots, t_n)$ is true if and only if the term $f_p(t_1, \ldots, t_n)$ evaluates to the distinguished constant $\bullet$.

14.5.2 Satisfiability Preservation

Teorema 14.5 (Satisfiability Preservation)

Let F be a quantifier-free formula and let F' be the result of applying predicate elimination to F. Then F is satisfiable if and only if F' is satisfiable in models with cardinality greater than 1.

Note. The restriction to models with cardinality greater than 1 is necessary to avoid trivial models. In a model with domain $D = \{d\}$ (cardinality 1), all terms evaluate to the same element d, making all equalities trivially true and potentially invalidating the transformation.

14.5.3 Example: The Need for Cardinality Restriction

Consider the following two formulas:

Original formula:

$$S_1 = \{\forall x \, \forall y \, (x = y), \neg P(a)\}$$

This formula is $\mathcal{T}_E$-satisfiable. We can construct a model with domain $D = \{d_1, d_2\}$ where:

- $\Phi(a) = d_1$.
- $\Phi(P) = \{d_2\}$ (so $d_1 \notin \Phi(P)$, making $\neg P(a)$ true)
- The first axiom $\forall x \, \forall y \, (x = y)$ is vacuously false in this model, but if we interpret it as requiring equality to be universal, we need a different construction

Actually, let's reconsider: $\{\forall x \, \forall y \, (x = y), \neg P(a)\}$ asserts that all elements are equal (domain has cardinality 1) and $P(a)$ is false. This is satisfiable in a model with $D = \{d\}$, $\Phi(a) = d$, and $\Phi(P) = \varnothing$.

Transformed formula:

$$S_2 = \{\forall x \, \forall y \, (x = y), f_P(a) \neq \bullet\}$$

In a model with cardinality 1, we have $D = \{d\}$, so $\Phi(f_P)(d) = d$ and $\Phi(\bullet) = d$. Therefore, $f_P(a) = \bullet$, contradicting $f_P(a) \neq \bullet$. Thus S_2 is unsatisfiable in models of cardinality 1.

This example demonstrates that the transformation preserves satisfiability only when we exclude trivial models with cardinality 1.

14.6 Treatment of Free Variables

In quantifier-free formulas, free variables may appear. For the purpose of satisfiability checking, free variables are treated analogously to constants.

Note. (Free Variables as Constants). To determine satisfiability of a formula F with

free variables $x_1, \ldots, x_k$, we need to find an interpretation $\mathcal{I}$ and an assignment β such that $\mathcal{I} \models_\beta F$.

Equivalently, we can treat each free variable x_i as a fresh constant symbol c_i and check satisfiability of the resulting closed formula. The formula is satisfiable if and only if there exists domain elements that can be assigned to these constants to satisfy the formula.

This observation allows us to reduce the satisfiability problem for formulas with free variables to the satisfiability problem for closed formulas by a simple syntactic substitution.

15 The Congruence Closure Algrithm

15.1 Motivating Example

Consider the following conjunction of literals:

$$x = g(x) \wedge f(x, g(g(x))) \neq g(x) \wedge f(x, x) = g(g(x))$$

Question: Is this formula satisfiable or unsatisfiable?

Intuitive analysis: This formula appears to be unsatisfiable. From $x = g(x)$, we can derive $g(x) = g(g(x))$ by applying the function g to both sides (congruence). Then from $f(x, x) = g(g(x))$ and the derived equality $g(x) = g(g(x))$, we obtain $f(x, x) = g(x)$. But we also have $x = g(x)$, so by congruence on f, we get $f(x, x) = f(x, g(x))$. However, we need to check whether $f(x, g(g(x))) = g(x)$ follows from the positive equalities, which would contradict $f(x, g(g(x))) \neq g(x)$.

15.2 The congruence closure Decision Procedure

Definizione 15.1 (Congruence Closure Algorithm)

The **congruence closure algorithm** is a decision procedure for satisfiability in the quantifier-free fragment of the theory of equality. The algorithm operates as follows:

1. **Build the smallest congruence**: Construct the smallest congruence relation that satisfies all postive equalities in the input formula
2. **Check negative equalities**: For each negative equality $s \neq t$ in the input, check whether s and t belong to the same equivalence class in the congruence
3. **Determine satisfiability**:
   - If any pair (s, t) with $s \neq t$ belongs to the same equivalence class, the formula is **unsatisfiable**
   - Otherwise, the congruence provides a model, and the formula is **satisfiable**

The key insight is that the smallest congruence containing all positive equalities is included in every congruence that satisfies those equalities. Therefore, if a negative equality is violeted in the smallest congruence, it must be violated in every model.

15.3 Equivalence Relations and Partitions

Before describing the algorithm in detail, we recall fundamental properties of equivalence relations.

Definizione 15.2 (Equivalence Relation)

An **equivalence relation** R on a set S is a binary relation that is:

- **Reflexive**: $\forall s \in S, sRs$.
- **Symmetric**: $\forall s, t \in S, sRt \Rightarrow tRs$.
- **Transitive**: $\forall s, t, u \in S, (sRt \wedge tRu) \Rightarrow sRu$.

Definizione 15.3 (Equivalence Class)

Given an equivalence relation R on set S, the **equivalence class** of an element $s \in S$ is:

$$[s]_R = \{s' \in S \mid s'Rs\}$$

Definizione 15.4 (Quotient Set)

The **quotient set** S/R is the set of all equivalence classes:

$$S/R = \{[s]_R \mid s \in S\}$$

Definizione 15.5 (Partition Property)

An equivalence relation R on S induces a parittion of S. That is, for any two distinct equivalence classes $[s]_R$ *and* $[p]_R$*:*

$$s \nsim_R p \quad \Rightarrow \quad [s]_R \cap [p]_R = \varnothing$$

where $s \backsim_R p$ *denotes sRp*

Conversely, every element belongs to exactly one equivalence class, and the union of all equivalence classes equals S.

This partition property is fundamental to the congruence closure algorithm, as it allows us to represent the congruence relation efficiently using a union-find data structure.

Parte IV
Decision Procedures for Quantifier-Free Theories

16 The Congruence Closure Algorithm

16.1 Computing Congruence Closure

We now describe how to compute the congruence closure for the quantifier-free fragment of the theory of equality $\mathcal{T}_E$.

16.1.1 Input Problem Structure

Consider an input formula F consisting of a conjunction of equality and disequality literals:

$$F = \underbrace{\{s_1 = t_1, \ldots, s_n = t_n\}}_{F^+} \cup \underbrace{\{s_{n+1} \neq t_{n+1}, \ldots, s_{n+m} \neq t_{n+m}\}}_{F^-}$$

where:

- F^+ denotes the set of **positive equalities** (equations)
- F^- denotes the set of **negative equalities** (disequations)

The congruence closure algorithm will construct the smallest congruence containing all equalities in F^+, then check whether any disequality F^- is violated.

16.1.2 Equivalence relations and Representatives

Given a binary equivalence relation $R \subseteq S \times S$ on a set S, we can construct its quotient:

$$S/R = \{[s]_R \mid s \in S\}$$

where the equivalence class of s is:

$$[s]_R = \{p \in S \mid pRs\}$$

The element s is called a **representative** of the equivalence class $[s]_R$.

16.1.3 Disjointness of Equivalence Classes

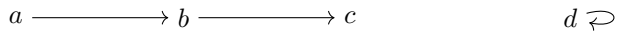
Definizione 16.1 (Disjoint Class)

For any two equivalence classes $[s_1]_R$ and $[s_2]_R$, either they are identical or the are disjoint:

$$[s_1]_R \cap [s_2]_R \neq \varnothing \quad \Rightarrow \quad [s_1]_R = [s_2]_R$$

Proof. Suppose there exists $p \in [s_1]_R \cap [s_2]_R$. Then:

- pRs_1 (since $p \in [s_1]_R$)



- pRs_2 (since $p \in [s_2]_R$)

By symmetry and transitivity of R, we have $s_1 R s_2$. Therefore, for any $q \in [s_1]_R$, we have qRs_1 and $s_1 R s_2$, so by transitivity qRs_2, which means $q \in [s_2]_R$. Similarly, $[s_2]_R \subseteq [s_1]_R$. Thus $[s_1]_R = [s_2]_R$.

16.1.4 Example: Equivalence Closure

Consider the binary relation:

$$R = \{(a, b), (b, c), (d, d)\}$$

We can visualize this as a directed graph: The **equivalence closure** R^* of R must include:

- **Reflexivity**: $(a, a), (b, b), (c, c), (d, d)$.
- **Symmetry**: $(b, a), (c, b)$.
- **Transitivity**: $(a, c), (c, a)$.

Thus:

$$R^* = \{(a, a), (a, b), (a, c), (b, a), (b, b), (b, c), (c, a), (c, b), (c, c)(d, d)\}$$

The quotient set is:

$$S/R^* = \{\{a, b, c\}, \{d\}\}$$

Definizione 16.2 (Equivalence Closure)

Given a binary relation R on set S, the **equivalence closure** of R, denoted R^*, is the $\subseteq$-smallest equivalence relation such that:

1. R^* is an equivalence relation
2. $R \subseteq R^*$

Equivalently, R^* is the intersection of all equivalence relations containing R.

The equivalence closure can be computed by iteratively applying reflexivity, symmetry, and transitivity rules until a fixed point is reached.

16.1.5 Understanding the $\subseteq$-Smallest Property

Definizione 16.3 (Minimality of Equivalence Closure)

Let R be a binary relation and R^* be its equivalence closure. For any equivalence relation P such that $R \subseteq P$, we have $R^* \subseteq P$.

This means that R^* is the **smallest** equivalence relation containing R with respect to the subset ordering $\subseteq$.

Definizione 16.4 (Congruence Closure)

Given a binary relation R, the **congruence closure** of R, denoted R^*_{cong} is the $\subseteq$-smallest relation such that:

1. R^*_{cong} is a congruence relation
2. $R \subseteq R^*_{cong}$

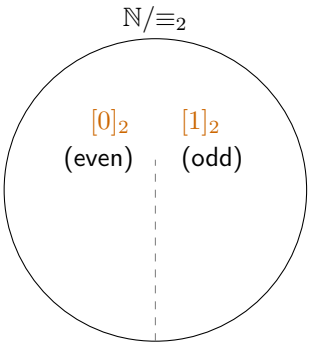
A congruence relation is an equivalence realtion that is preserved under function application: if $s_1 \backsim t_1, \ldots, s_n \backsim t_n$, then $f(s_1, \ldots, s_n) \backsim f(t_1, \ldots, t_n)$ for all function symbols f.

16.1.6 Example: Refinement of Equivalence Relations

To illustrate the concept of one equivalence relation being smaller (finer) than another, consider the following example on the natural numbers $\mathbb{N}$.

Equivalence modulo 2: Define the relation $\equiv_2 \subseteq \mathbb{N} \times \mathbb{N}$ by:

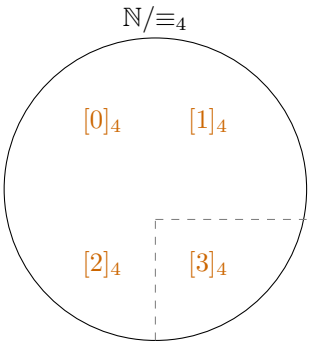
$$n \equiv_2 m \quad \text{iff} \quad n \equiv m \pmod 2$$



Equivalence modulo 4: Define the relation $\equiv_4 \subseteq \mathbb{N} \times \mathbb{N}$ by:

$$n \equiv_4 m \quad \text{iff} \quad n \equiv m \pmod 4$$

This relation partitions $\mathbb{N}$ into four equivalence classes:



Definizione 16.5 (Refinement)

*The relation $\equiv_4$ is a **refinement** of $\equiv_2$. That is:*

$$n \equiv_4 m \quad \Rightarrow \quad n \equiv_2 m$$

However, the converse does not hold:

$$n \equiv_2 m \quad \not\Rightarrow \quad n \equiv_4 m$$

Proof. If $n \equiv_4 m$, then $n \equiv m \pmod 4$, which means $n - m = 4k$ for some $k \in \mathbb{Z}$. Therefore, $n - m = 2(2k)$, so $n \equiv m \pmod 2$, i.e., $n \equiv_2 m$.

For the converse, consider $n = 0$ and $m = 2$. Then $n \equiv_2 m$ (both are even), but $n \not\equiv_4 m$ (since $0 \equiv 0 \pmod 4$ and $2 \equiv 2 \pmod 4$).

Set-theoretic interpretation: This means that $(n, m) \in \equiv_4 \Rightarrow (n, m) \in \equiv_2$, so:

$$\equiv_4 \subseteq \equiv_2$$

In this sense, $\equiv_4$ is **strictly smaller** than $\equiv_2$ (it is a proper subset), and provides a finer partition of $\mathbb{N}$.

16.2 The Congruence Closure Decision Procedure

We now return to the main problem: deciding satisfiability for quantifier-free formula in the theory of equality.

16.2.1 Algorithm Description

Recall that the input formula F has the form:

$$F = \underbrace{\{s_1 = t_1, \ldots, s_n = t_n\}}_{F^+} \cup \underbrace{\{s_{n+1} \neq t_{n+1}, \ldots, s_{n+m} \neq t_{n+m}\}}_{F^-}$$

The congruence closure algorithm proceeds as follows: **Step 1**: Compute $\mathrm{CC}(F^+)$,

Algorithm 1 Congruence Closure Decision Procedure

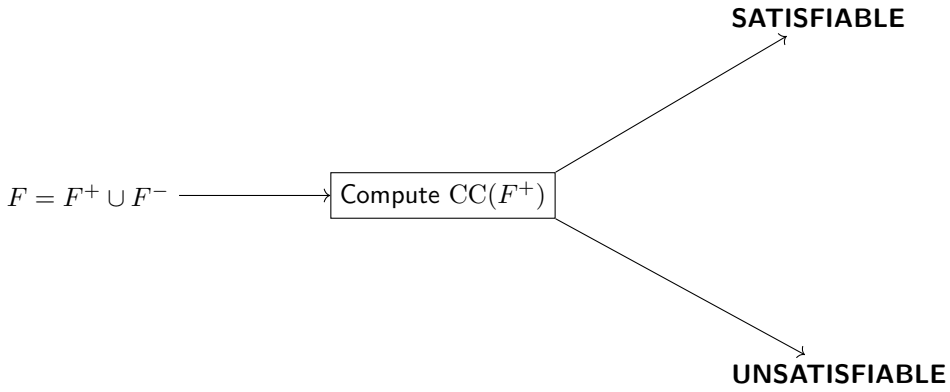
Require: Formula $F = F^+ \cup F^-$
1: Compute the congruence closure $\mathrm{CC}(F^+)$ of the positive equalities
2: **for** $i = n + 1$ **to** $n + m$ **do**
3: **if** $(s_i, t_i) \in \mathrm{CC}(F^+)$ **then**
4: **return** UNSATISFIABLE
5: **end if**
6: **end for**
7: **return** SATISFIABLE

the smallest congruence relation containing all pairs (s_i, t_i) for $i = 1, \ldots, n$.

Step 2: For each disequality $s_i \neq t_i$ in F^- (where $n + 1 \leq i \leq n + m$), check whether (s_i, t_i) belongs to the same equivalence class in $\mathrm{CC}(F^+)$.

Step 3:

- If $(s_j, t_j) \in \mathrm{CC}(F^+)$ for some j with $n+1 \leq j \leq n+m$, then F is **unsatisfiable**
- Otherwise, F is **satisfiable**



16.2.2 Correctness of the Algorithm

Teorema 16.6 (Soundness and Completeness)

The congruence closure algorithm is sound and complete for deciding satisfiability in the quantifier-free fragment of $\mathcal{T}_E$.

Proof sketch. **Soundness (Unsatisfiability)**: Suppose there exists j with $n+1 \leq j \leq n+m$ such that $(s_j, t_j) \in \mathrm{CC}(F^+)$

Since $\mathrm{CC}(F^+)$ is the smallest congruence relation containing F^+, the pair (s_j, t_j) must belong to *every* congruence relation that contains F^+.

Therefore, in any model $\mathcal{I}$ that satisfies all equalities in F^+, we must have $\mathcal{I} \models s_j = t_j$. But F also contains $s_j \neq t_j$, so $\mathcal{I} \not\models s_j \neq t_j$. Thus, no model can satisfy both F^+ and F^-, making F unsatisfiable.

Completeness (Satisfiability): If $(s_i, t_i) \notin \mathrm{CC}(F^+)$ for all i with $n+1 \leq i \leq n+m$, then we can contruct a model by interpreting the equivalence classes of $\mathrm{CC}(F^+)$ as domain elements. This model satisfies all equalities in F^+ (by construction) and all disequalities in F^- (since the terms belong to different equivalence classes).

The key insight is that the congruence closure captures exactly those equalities that are *forced* by the positive equalities in F^+ together with the congruence axioms of equality.

16.3 Examples: Manual Execution

We now illustrate the congruence closure algorithm with concrete Examples

16.3.1 Notation and Setup

For a formula F on the form:

$$F = \{s_1 = t_1, \ldots, s_n = t_n\} \cup \{s_{n+1} \neq t_{n+1}, \ldots, s_{n+m} \neq t_{n+m}\}$$

we define S_F as the **set of all terms** that appear in F.

16.3.2 Example 1

Consider the formula:

$$F = \{x = g(x), f(x, x) = g(g(x)), f(x, g(g(x))) \neq g(x)\}$$

The set of terms is:

$$S_F = \{x, g(x), f(x, x), g(g(x)), f(x, g(g(x)))\}$$

Notation:

- x is a **free variable symbol**
- f is a **binary function symbol** (arity 2)
- g is a **unary function symbol** (arity 1)

Positive equalities: $F^+ = \{x = g(x), f(x, x) = g(g(x))\}$.

Negative equalities: $F^- = \{f(x, g(g(x))) \neq g(x)\}$.

16.3.3 Example 2

Consider the formula:

$$F = \{f(a, b) = a, f(f(a, b), b) \neq a\}$$

The set of terms is:

$$S_F = \{f(a, b), a, b, f(f(a, b), b)\}$$

Notation:

- a, b are **constant symbols** (by convention)
- f is a **binary function symbol**

Positive equalities: $F^+ = \{f(a, b) = a\}$.

Negative equalities: $F^- = \{f(f(a, b), b) \neq a\}$.

Note (Free Varialbes vs. Constants). In the quantifier-free fragment, all variables are free. From a semantic perspective, free variables and constants can be used interchangeably: both are interpreted as arbitraty elements of the domain. The distinction is purely syntactic.

16.3.4 Step-by-Step Execution for Example 1

We now execute the congruence closure algorithm on Example 1:

$$F = \{x = g(x), f(x, x) = g(g(x)), f(x, g(g(x))) \neq g(x)\}$$

Step 1: Initialize the partition

Create a partition of S_F where every term is in its own singleton equivalence class:

$$\{\{x\}, \{g(x)\}, \{f(x, x)\}, \{f(x, g(g(x)))\}\}$$

Step 2: Process positive equalities

For each equality $(s_i = t_i) \in F^+$ (for all i with $1 \leq i \leq n$):

- Unite the equivalence class of s_i with the equivalence class of t_i.
- Unite the classes of all terms in S_F that become congruent as a consequence of $s_i = t_i$.

Processing $x = g(x)$:

Merge the classes $\{x\}$ and $\{g(x)\}$:

$$\{\{x, g(x)\}, \{f(x, x)\}, \{g(g(x))\}, \{f(x, g(g(x)))\}\}$$

Since $x = g(x)$, by congruence we derive $g(x) = g(g(x))$ (applying g to both sides). Merge these classes:

$$\{\{x, g(x), g(g(x))\}, \{f(x, x)\}, \{f(x, g(g(x)))\}\}$$

Processing $f(x, x) = g(g(x))$:

Since $g(g(x)) \in \{x, g(x), g(g(x))\}$ and $f(x, x) \in \{f(x, x)\}$, merge these classes:

$$\{\{x, g(x), g(g(x)), f(x, x)\}, \{f(x, g(g(x)))\}\}$$

Now check for congruence: since $x \sim g(g(x))$ (both in the same class), we have:

$$f(x, x) \sim f(x, g(g(x)))$$

by the congruence property (both arguments are in the same equivalence classes). Merge these classes:

$$\{\{x, g(x), g(g(x)), f(x, x), f(x, g(g(x)))\}\}$$

Step 3: Check negative equalities

Check whether $(s_i, t_i) \in \mathrm{CC}(F^+)$ for all i with $n + 1 \leq i \leq n + m$.

For the disequality $f(x, g(g(x))) \neq g(x)$:

- $f(x, g(g(x))) \in \{x, g(x), g(g(x)), f(x, x), f(x, g(g(x)))\}$.
- $g(x) \in \{x, g(x), g(g(x)), f(x, x), f(x, g(g(x)))\}$

Both terms are in the same equivalence class, so $(f(x, g(g(x))), g(x)) \in \mathrm{CC}(F^+)$.

Conclusion: The formula F is **UNSATISFIABLE**.

17 Implementation of Congruence Closure on a DAG

17.1 DAG Representation

The congruence closure algorithm can be implemented efficiently using a **Directed Acyclic Graph (DAG)** representation of the terms.

Consider the formula:

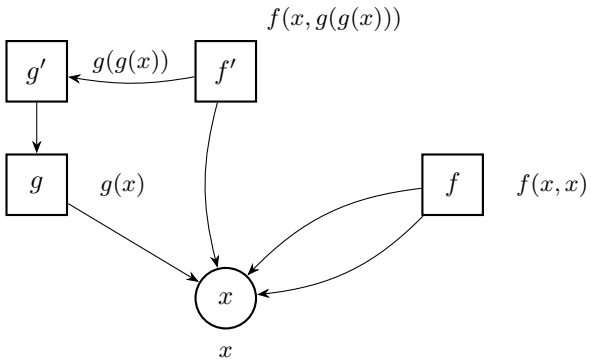
$$F = \{x = g(x), f(x, x) = g(g(x))\} \cup \{f(x, g(g(x))) \neq g(x)\}$$

The set of terms is:

$$S_F = \{x, g(x), g(g(x)), f(x, x), f(x, g(g(x)))\}$$

We represent these terms as a graph where:

- Each term is a node
- Edges connect function symbols to their arguments
- We use g and g' to distinguish the two applications of function g
- We use f and f' to distinguish the two applications of function f



The graph shoes the structure of the terms:

- Node g has and edge to x (representing $g(x)$)
- Node f has two edges to x (representing $f(x, x)$)
- Node g' has and edge to g (representing $g(g(x))$)
- Node f' has edges to x and g' (representing $f(x, g(g(x)))$)

This is a graph representation of the term structure, which allows efficient implementation of the congruence closure algorithm using union-find data structures to track equivalence classes of nodes.

Definizione 17.1 (Node Data Structure)

Each mode in the DAG is represented by a data structure with the following fields:

```
type node = {
  id: Id;
  fn: String;
  args: Id list;
  mutable find: Id;
  mutable ccpar: Id set;
}
```

For a node n of type `node`, the fields have the following meaning:

- **n.id**: The unique identifier of node n
- **n.fn**: The function symbol (or variable name) represented by this node
- **n.args**: The list of identifiers of the argument nodes (empty for variables and constants)
- **n.find**: The identifier of the representative node of the equivalence class to which n belongs
  - *Initialization*: `n.find = n.id` (each node starts in its own equivalence class)
- **n.ccpar**: The set of identifiers of the parent nodes of n (nodes that have n as an argument) and all nodes congruent to n

This data structure supports the union-find operations needed for the congruence closure algorithm, where `find` implements the path to the representative of each equivalence class, and `ccpar` tracks congruence relationships for efficient propagation.

17.2 Basic Operations

We now define the basic operations on the DAG structure.

17.2.1 The NODE Function

The function `NODE` retrives a node given its identifier:

$$\texttt{NODE} : \texttt{Id} \to \texttt{Node}$$

$$\texttt{NODE}(i) = n \quad \text{such that} \quad n.\texttt{id} = i$$

17.2.2 The FIND Operation

The `FIND` function takes an identifier and returns the identifier of the representative of the equivalence class to which the node belongs:

$$\texttt{FIND} : \texttt{Id} \to \texttt{Id}$$

```
let rec FIND i =
  let n = NODE i in
    if n.find = i then i
    else FIND n.find
```

This recursive function follows the `find` pointers until it reaches a node that is its own representative (i.e., $n.\texttt{find} = n.\texttt{id}$).

17.2.3 The UNION Operation

The `UNION` function merges two equivalence classes:

$$\texttt{UNION} : \texttt{Id} \times \texttt{Id} \to \texttt{void}$$

```
let UNION i_1 i_2 =
  let n_1 = NODE (FIND i_1) in
  let n_2 = NODE (FIND i_2) in
  n_1.find <- n_2.find;
  n_2.ccpar <- n_2.ccpar U n_1.ccpar;
  n_1.ccpar <- empty_set
```

The `UNION` operation performs the following steps:

1. Find the representatives n_1 and n_2 of the two equivalence classes
2. Make n_1 point to n_2 (merging the classes)
3. Transfer the `ccpar` set from n_1 to n_2
4. Clear the `ccpar` set of n_1

Note. During execution, $n.\texttt{ccpar} \neq \varnothing$ if and only if n is a representative (i.e., $n.\texttt{find} = n.\texttt{id}$). This invariant ensures that only representative nodes maintain the parent and congruence information.

17.2.4 The CCPAR Function

The `CCPAR` function return the `ccpar` set of the representative of a given node:

$$\texttt{CCPAR} : \texttt{Id} \to \texttt{Id set}$$

```
let CCPAR i = (NODE (FIND i)).ccpar
```

This function first finds representative of node i, then returns its `ccpar` set.

17.2.5 The CONGRUENT Function

The `CONGRUENT` function checks whether two nodes are congruent (i.e., they have the same function symbol and their corresponding arguments are in the same equivalence classes):

$$\texttt{CONGRUENT} : \texttt{Id} \times \texttt{Id} \to \texttt{bool}$$

```
let CONGRUENT i_1 i_2 =
  let n_1 = NODE i_1 in
  let n_2 = NODE i_2 in
  n_1.fn = n_2.fn &&
  |n_1.args| = |n_2.args| &&
  forall i in {1, ..., |n_1.args|}
    FIND n_1.args[i] = FIND n_2.args[i]
```

Two nodes are congruent if:

1. They have the same function symbol $n_1.\texttt{fn} = n_2.\texttt{fn}$
2. They have the same number of arguments ($|\, n_1.\texttt{args} \,| \;=\; |\, n_2.\texttt{args} \,|$)
3. Their corresponding arguments are in the same equivalence classes

17.2.6 The MERGE Function

Definizione 17.2 (MERGE Operation)

The `MERGE` function merges two equivalence classes and recursibely propagates congruence:

$$\texttt{MERGE} : \texttt{Id} \times \texttt{Id} \to \texttt{void}$$

```
let rec MERGE i_1 i_2 =
  if FIND i_1 =/= FIND i_2 then
    begin
      let P_1 = CCPAR i_1 in
      let P_2 = CCPAR i_2 in
      UNION i_1 i_2;
      foreach (t_1, t_2) in P_1 x P_2 do
        if FIND t_1 =/= FIND t_2 && CONGRUENT t_1 t_2
        then MERGE t_1 t_2
      done
    end
```

The `MERGE` operation works as follows:

1. Check if i_1 and i_2 are already in the same equivalence class; if so, do nothing
2. Save the ccpar sets P_1 and P_2 of both nodes
3. Perform `UNION` to merge the equivalence classes
4. For each pair (t_1, t_2) in the Cartesian product $P_1 \times P_2$:
   - If t_1 and t_2 are not already equivalent and are congruent
   - Recursively merge them with `MERGE`

This recursive propagation ensures that when two nodes become equivalent, all pairs of congruent parent nodes are also merged, maintaining the congruence closure property

17.3 The Complete Congruence Closure Algorithm

We can now describe the complete congruence closure algorithm using the DAG-based implementation

17.3.1 Algorithm

Teorema 17.3 (Complete Congruence Closure Algorithm (DAG-based))

Given a formula:

$$F = \{s_1 = t_1, \ldots, s_n = t_n\} \cup \{s_{n+1} \neq t_{n+1}, \ldots, s_{n+m} \neq t_{n+m}\}$$

The algorithm proceeds as follows:

Algorithm 2 Complete Congruence Closure Algorithm (DAG-based)

Require: Formula $F = F^+ \cup F^-$ where $F^+ = \{s_1 = t_1, \ldots, s_n = t_n\}$ and $F^- = \{s_{n+1} \neq t_{n+1}, \ldots, s_{n+m} \neq t_{n+m}\}$
Construct the DAG for F
for $i = 1$ to n **do do**
  $\texttt{MERGE}(s_i, t_i)$
end for
for $i = n + 1$ to $n + m$ **do do**
  if $\texttt{FIND}(s_1) = \texttt{FIND}(t_i)$ **then then**
    return UNSATISFIABLE
  end if
end for
return SATISFIABLE

Steps:

1. **Construct the DAG**: Build the DAG representation of all terms in F
2. **Process equalities**: For each equality $s_i = t_i$ in F^+, call $\texttt{MERGE}(s_i, t_i)$ to merge the equivalence classes and propagate congruence
3. **Check disequalities**: For each disequality $s_i \neq t_i$ in F^-, check if s_i and t_i, are in the same equivalence class (i.e., $\texttt{FIND}(s_i) = \texttt{FIND}(t_i)$)
   - If yes, return UNSATISFIABLE
4. **Return result**: If no conflicts found, return SATISFIABLE

17.3.2 Complexity

Definizione 17.4 (Complexity Analysis)

Let n be the number of nodes in the DAG. The complexity of the congruence closure algorithm is $O(n^2)$ in the worst case, where the quadratic factor comes from the potential number of congruence propagations that may occur during the $\texttt{MERGE}$ operations.

18 Worked Examples of Congruence Closure

In this section, we present detailed worked examples demonstrating the execution of the congruence closure algorithm on concrete formulas. These examples illustrate the DAG construction, the initialization of node fields, and the step-by-step exectuion of the `MERGE` operations.

18.1 Example 1: Simple Commutativity

Esempio 18.1 (Congruence Closure with Commutativity)

Consider the formula without quantifiers:

$$F = \{f(x, y) = f(y, x)\} \cup \{f(a, y) = f(y, a)\}$$

where a is a constant symbol.

Step 1: Identify the set of subterms

The set of all subterms appearing in F is:

$$S_F = \{x, y, a, f(x, y), f(y, x), f(a, y)f(y, a)\}$$

Step 2: Construct the DAG

We construct a DAG where each node represents a unique subterm. Each node has the following fields:

- **id**: A unique integer identifier
- **fn**: The function symbol (or variable/constant name) as a string
- **args**: List of identifiers of arugment nodes (empty for variables/constants)
- **find**: Initially set to the node's own id
- **ccpar**: Set of parent node identifiers (initially contains all nodes that have this node as an argument)

The DAG structure ca ben visualized as:

$$\begin{array}{cccc} f_3 & f_4 & f_5 & f_6 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ (x_0, y_1) & (y_1, x_0) & (a_2, y_1) & (y_1, a_2) \end{array}$$

Step 3: Initialize node fields

Definizione 18.2 (Initial DAG Node Configuration)

Node f_3: `id` $= 3$, `fn` $=$ "f", `args` $= [0, 1]$, `find` $= 3$, `ccpar` $= \{\}$

Node x_0: `id` $= 0$, `fn` $=$ "x", `args` $= []$, `find` $= 0$, `ccpar` $= \{3, 4\}$

Node f_4: id = 4, fn = "f", args = [1, 0], find = 4, ccpar = $\{\}$

Node y_1: id = 1, fn = "y", args = [], find = 1, ccpar = $\{3, 4, 5, 6\}$

Node f_5: id = 5, fn = "f", args = [2, 1], find = 5, ccpar = $\{\}$

Node a_2: id = 2, fn = "a", args = [], find = 2, ccpar = $\{5, 6\}$

Node f_6: id = 6, fn = "f", args = [1, 2], find = 6, ccpar = $\{\}$

Key observation about initialization:

- Each node's `find` intially points to itself
- The `ccpar` field of each leaf node (variables/constants) contains the ids of all funcion nodes that have it as an argument
- Function nodes initially have empty `ccpar` sets

Step 4: Process the equality $f(x, y) = f(y, x)$

We exectue `MERGE`(3,4) corresponding to the equality $f(x, y) = f(y, x)$:

1. Check if `FIND`(3) $\neq$ `FIND`(4): YES (they are 3 and 4 respectively)
2. Save parent sets: $P_1 = \texttt{CCPAR}(3) = \varnothing, P_2 = \texttt{CCPAR}(4) = \varnothing$
3. Execute `UNION`(3, 4):
   - Set node 4's `find` field to 3
   - Merge `ccpar` sets (both empty in this case)
4. Check Cartesian products $P_1 \times P_2 = \varnothing \times \varnothing = \varnothing$
5. No further merges needed

After this operation, nodes f_3 and f_4 are in the same equivalence class.

Step 5: Check the disequality (if any)

In this example, we only have equalities. The formula $f(a, y) = f(y, a)$ represents another equality that could be processed similarly. Since $f(a, y)$ and $f(y, a)$ (nodes 5 and 6) remain in distinct equivalence classes after processing the first equality, the formula is **SATISFIABLE** - there is no contradiction.

18.2 Example 2: Cyclic Function Application with Disequality

Esempio 18.3 (Detecting Unsatisfiability through Congruence Closure)

Consider the formula:

$$F = \{f^3(a) = f^5(a)\} \cup \{f(a) \neq a\}$$

where we use the notation $f^n(a)$ to denote n applications of function f to

constant a. Explicity:

- $f^3(a) = f(f(f(a)))$
- $f^5(a) = f(f(f(f(f(a)))))$

Step 1: Identify the set of subterms

The set of all subterms is:

$$S_F = \{a, f(a), f^2(a), f^3(a), f^4(a), f^5(a)\}$$

Step 3: Construct the DAG

We build a chain-like DAG where each node represents successive applications of f:

$$f^5(a) \to f^4(a) \to f^3(a) \to f^2(a) \to f(a) \to a$$

Step 3: Initialize node fields

Definizione 18.4 (Initial DAG Configuration)

Node 5: id $= 5$, fn $=$ "f", args $=$ [4], find $= 5$, ccpar $= \{\}$
Node 4: id $= 4$, fn $=$ "f", args $=$ [3], find $= 4$, ccpar $= \{5\}$
Node 3: id $= 3$, fn $=$ "f", args $=$ [2], find $= 3$, ccpar $= \{4\}$
Node 2: id $= 2$, fn $=$ "f", args $=$ [1], find $= 2$, ccpar $= \{3\}$
Node 1: id $= 1$, fn $=$ "f", args $=$ [0], find $= 1$, ccpar $= \{2\}$
Node 0: id $= 0$, fn $=$ "a", args $=$ [], find $= 0$, ccpar $= \{1\}$

Step 4: Process the equality $f^3(a) = f^5(a)$

The equality $f^3(a) = f^5(a)$ implies that after three applications of f, we get the same result as after five applications. This creates a cycle in the equivalence structure.

We execute MERGE(3,5). However, the congruence closure algorithm will propagate this equality through the structure, ultimately establishing relationships between multiple nodes.

Logical Implications of the equality:

From $f^3(a) = f^5(a)$, by applying the congruence rule repeatedly, we can derive:

$$f^3(a) = f^5(a)$$
$$f^4(a) = f^6(a) = f(f^5(a)) = f(f^3(a))$$
$$f^2(a) = f^4(a) \text{ (by applying } f^{-1} \text{ conceptually)}$$
$$f(a) = f^3(a)$$
$$a = f^2(a)$$

More precisely, the algorithm establishes these equivalences through congruence propagation.

Step 5: Detailed execution trace

We now trace the execution of MERGE(3,5) and observe how congruence propagates:

Teorema 18.5 (Execution Trace)

Call: MERGE(3,5)

Since $f^3(a) = f^5(a)$, we need to merge nodes 3 and 5.

1. FIND(3) = 3, FIND(5) = 5 (different, so proceed)
2. $P_1 = \texttt{CCPAR}(3) = \{4\}, P_2 = \texttt{CCPAR}(5) = \varnothing$
3. UNION(3,5): merge the equivalence classes
4. Check $P_1 \times P_2 = \{4\} \times \varnothing = \varnothing$ (no propagation yet)

However, the equality $f^3(a) = f^5(a)$ implies $f^2(a) = f^4(a)$ by the congruence property.

This triggers further merges.

Call: MERGE(3, 0) (establishing $f^3(a) = a$)

1. $P_1 = \texttt{CCPAR}(3) = \{4\}, P_2 = \texttt{CCPAR}(0) = \{1\}$
2. UNION(3, 0)
3. Check $P_1 \times P_2 = \{4\} \times \{1\} = \{(4, 1)\}$
4. CONGRUENT(4,1)?
   - Same symbol: YES ($f = f$)
   - Same arity: YES (both unary)
   - Arguments equivalent: FIND(3) = FIND(0) after union: YES
5. Result: TRUE $\Rightarrow$ call MERGE(4,1)

Call: MERGE(4, 1) (establishing $f^4(a) = f(a)$)

1. $P_1 = \texttt{CCPAR}(4) = \{5\}, P_2 = \texttt{CCPAR}(1) = \{2\}$
2. UNION(4, 1)
3. Check $P_1 \times P_2 = \{5\} \times \{2\} = \{(5, 2)\}$
4. CONGRUENT(5,2)?
   - Same symbol: YES

- Same arity: YES
- Arguments equivalent: FIND(4) = FIND(1): YES

5. Result: TRUE ⇒ call MERGE(5,2)

Call: MERGE(5, 2) (establishing $f^5(a) = f^2(a)$)

1. $P_1 = \texttt{CCPAR}(5) = \varnothing, P_2 = \texttt{CCPAR}(2) = \{3\}$
2. UNION(5, 2)
3. Check $P_1 \times P_2 = \varnothing \times \{3\} = \varnothing$
4. No further propagation

The congruence closure process continues, eventualy establishing that all nodes $\{0, 1, 2, 3, 4, 5\}$ are in the same equivalence class, meaning:

$$a = f(a) = f^2(a) = f^3(a) = f^4(a) = f^5(a)$$

Step 6: Check the disequality $f(a) \neq a$

After processing all equalities through congruence closure, we check the disequality constraint:

- We have $f(a) \neq a$ in F^-
- Check: FIND(1) = FIND(0)?
- After congruence closure: YES (they are in the same equivalence class)
- **Conclusion**: The disequality is violated!

Result: The formula is **UNSATISFIABLE**. The quality $f^3(a) = f^5(a)$ forces a cycle that ultimately implies $f(a) = a$, which contradicts the disequality constraint $f(a) \neq a$.

19 Complexity Improvements and Optimiazions

The basic congruence closure algorithm presented earlier has complexity $O(n^2)$ for n nodes in the DAG. However, several optimizations have been developed to improve the practical and theoretical performance of the algorithm.

19.1 Hisotical Development

19.1.1 Nelson-Oppen Algorithm (1980)

The original congruence closure algorithm by Nelson and Oppen (1980) operates on a graph $G = (V, E)$ where:

- $| V | = n$ (number of nodes/vertices)

- $\mid E \mid = e$ (number of edges)

Definizione 19.1 (Nelson-Oppen Complexity)

The Nelson-Oppen algorithm has complexity $O(n^2)$ for $O(n)$ calls to `MERGE`

This quadratic complexity in the number of edges can be prohibitive for large formulas with many equalities.

19.1.2 Downey-Sethi-Tarjan Improvements (1980-1982)

Downey, Sethi and Tarjan developed several optimiazions that significantly improved the complexity of congruence closure Algorithms

Definizione 19.2 (Improved Complexity)

The Downey-Sethi-Tarjan optimizations reduce the complexity to $O(eloge)$ for $O(n)$ calls to `MERGE`

This represent a substantial improvement, especially for sparse graphs where e is much smaller than n^2.

19.2 Key Optimization

The improvements introduced by Downey, Sethi and Tarjan focus on two main aspects of the algorithm:

19.2.1 Optimiazion 1: Avoiding Recursion in FIND

Original recursive implementation:

```
let rec FIND i =
  let n = NODE i in
    if n.find = i then i
    else FIND n.find
```

Optimized non-recursive implmentation

```
let FIND i = (NODE i).find
```

Note. This optimization is correct only if the `UNION` operation is modified appropriately. When node i_1 becomes the representative of the merged equivalence class, the `find` field of **all nodes** in i_2's class must be updated to point directly to i_1.

This eliminates the need for path trasversal during `FIND` operations, reducing the cost from $O(h)$ (where h is the height of the union-find tree) to $O(1)$.

19.2.2 Optimiazion 2: Union by Size Heuristic

When mergin two equivalence classes via `UNION`(i_1, i_2), we must choose which representative becomes the representative of the merged class. Two effective heuristic are:

Definizione 19.3 (Union Heuristic)

Heuristic 1: Union by Equivalence Class Size

Choose as the representative of the new class the representative of the **larger equivalence class:**

- Compare $|[i_1]|$ and $|[i_2]|$ (sizes of the equivalence classes)
- Make the representative of the larger class the new representative

Heuristic 2: Union by CCPAR Size

Choose as the representative the node whose `ccpar` field has **more members**:

- Compare $|i_1.\texttt{ccpar}|$ and $|i_2.\texttt{ccpar}|$
- Make the representative with the larger `ccpar` set the new representative

Rationale: These heuristics ensure that the union-find tree remains balanced, preventing degenerate cases where the tree becomes a long chain. By always attaching the samller structure to the larger one, we maintain logarithmic depth in the union-find structure.

19.3 Implementation Note

The `CCPAR` function must retrive the `ccpar` set of the representative node:

```
let CCPAR i = (NODE (FIND i)).ccpar
```

This ensure that we always access the `ccpar` information from the current representative of the equivalence class, which is essential for correctness after multiple `UNION` operations.

19.4 Practical Impact

These optimiazions have significant practical impact:

- **Reduced time complexity**: From $O(e^2)$ to $O(eloge)$
- **Better cache locality**: Non-recursive `FIND` reduces function call overhead
- **Balanced structures**: Union heuristics prevent worst-case tree depths
- **Scalability**: Enables congruence closure on much larger formulas

These improvements make congruence closure practical for real-world applications in automated theorem proving, program verification, and satisfiability modulo theories (SMT) solvers.